

8 Classes and objects - the basics

Learning objectives

- Understand what a class and an object are, and know the difference between a class and an instance
- Define a class with `__init__()`, instance attributes and instance methods
- Understand what `self` is and how Python passes it automatically
- Use dot notation to access attributes and call methods
- Assess when something should become a class and when a function is enough
- Understand Python's convention for "private" attributes with a leading underscore
- Implement `__str__()` for readable output of objects
- Design classes that collaborate - such as `Account` and `Transaction`
- Use properties with `@property` and `@name.setter` for access control and validation
- Understand what class variables and class methods are, and distinguish between `@classmethod` and `@staticmethod`
- Implement common dunder methods for operator overloading: `__eq__()`, `__lt__()`, `__gt__()`, `__getitem__()`, `__contains__()`
- Implement `__bool__()` to control the truthiness of custom objects
- Understand the difference between overloading and overriding
- Use `@dataclass` to reduce boilerplate for simple data classes, including `field()` with `default_factory` for mutable default values

8.1 Introduction

In Python and other languages, *classes* are a way to create our own data types, combining data (attributes) and functionality (methods) in one structure. An *object* is an *instance* of a class - a concrete occurrence.

We have already worked with built-in classes such as `int`, `str` and `list`. These were created by the Python developers in the same way as we will now create our own.

`int` is a relatively simple class that can store an integer and perform operations on it.

`list` is far more advanced - it can store a collection of elements, sort them, iterate over them and much more.

We create objects of built-in types like this:

```
an_int = 1          # Python infers that this is int from
context
an_int = int(1)     # Explicit use of the constructor
```

For our own classes we must always use the constructor (explained below) explicitly - Python cannot infer the type from context the way it does for `int` and `str`.

We start with simple, conceptual examples and gradually build up to more advanced classes.

8.2 Creating a class

A class is a template or blueprint for creating objects. We define what data (attributes) and what behaviour (methods) the objects will have. Each object created from the class gets its own values for the attributes.

```
1: # file: sc_08_01_Car.py
2: class Car:
3:     def __init__(self, brand, color):
4:         self._brand = brand
5:         self._color = color
6:
7:     def drive(self):
8:         print(f"{self._brand} car is driving!")
9:
10:    def show_info(self):
11:        print(f"Car: {self._brand}, Color: {self._color}")
12:
13: my_car = Car("Toyota", "Blue")
14: another_car = Car("Volvo", "Red")
15:
16: my_car.drive()
17: my_car.show_info()
18: another_car.show_info()
```

Output:

```
Toyota car is driving!
Car: Toyota, Color: Blue
```

```
Car: Volvo, Color: Red
```

Code comments:

Line 2

The class name starts with a capital letter - this is a Python convention. When multiple words are used, each word is capitalised (PascalCase), for example BankAccount or CarFactory.

Line 3-5

`__init__()` is the constructor - it is called automatically when we create an object (lines 13-14). The parameter `self` is a reference to the object itself. The attributes `_brand` and `_color` are created on the object with `self._name`. *A single underscore signals that they are intended for internal use.*

Line 7-11

The instance methods `drive()` and `show_info()` take `self` as the first parameter so they have access to the object's own attributes.

Line 13-14

We create two Car objects. The arguments in parentheses are passed to `__init__()` as `brand` and `color`. Python inserts `self` automatically.

What exactly is the Car class that we have created?

We have simply tried to model something *that exists in the real world* - a car. Admittedly, our first attempt may seem rather silly, and it obviously cannot be used for anything useful in this version.

We chose `Car` as the first example because it is something recognizable. It also lets us demonstrate the difference between what a `Car` *can do* - its methods, such as `drive()` - and what *properties* it has - its attributes, such as `_brand` and `_color`.

Not all classes that we will create will be as concrete as `Car`. Often, we need classes that are more abstract, designed to collect functionality and properties into one unit.

8.3 `__init__()` - the constructor

`__init__()` is called automatically when an object is created. It initialises the object's attributes and ensures the object is in a valid state from the very start.

Attributes should always be created in `__init__()`, not scattered around other methods. This gives a clear overview and predictability - and avoids bugs where a method tries to use an attribute that has not been set yet. Attributes are created with `self._name` and live as long as the object exists.

In Python, this is primarily a style rule, not a technical requirement. An attribute created in another method is just as much an instance attribute as an attribute created in `__init__()`. The important point is that the attribute is created through `self`.

This is a major difference from languages such as C++ and Java, where fields are usually declared explicitly in the class definition before they are used. In Python, an object's attributes can appear while the program is running. This gives flexibility, but also more responsibility: if attributes are created in different methods, it becomes harder to see which state the object may have. Therefore, we should always create the object's fixed attributes in `__init__()`.

8.4 What is self?

To understand `self` it is important to distinguish between two things:

- The *code* defined in a class is *shared by all objects*.
- The *attributes* - the data - are *unique to each object*.

When we write `my_car.drive()` Python rewrites it as `Car.drive(my_car)` behind the scenes. That is how `self` gets its value; - Python passes a reference to the current object as the first argument. The same happens at creation:

```
my_car = Car("Toyota", "Blue")
# Becomes:
Car.__init__(my_car, "Toyota", "Blue")
```

But wait - how can we pass `my_car` in when the object has not been created yet? The answer is that Python does this in two steps: first the method `__new__()` is called, which allocates memory and creates the object itself, then `__init__()` is called to initialise it. By the time `__init__()` is called, the object already exists, and `self` points to it.

8.5 Dot notation: .

The dot notation tells Python that we want to use something that belongs to an object - either a value (attribute) or a function (method). Without the dot Python does not know which object we mean.

When we write `my_car.drive()` we are asking the object `my_car` to run the code for `drive()`, applied to its own attributes.

8.6 When should something become a class?

When you model something that has both data and behaviour that naturally belong together, it is probably a good candidate for a class. The right question to ask is: does what I am trying to model have both data and behaviour that go together?

Create a class when:

- You generally need to model something in the real world (at least, this would be a *candidate* to become a class)
- You have several related pieces of data that belong together
- You need functions that operate on this data
- You need to create multiple instances with the same structure but different content
- You want to organise the code in a clean and reusable way

Not everything needs to become a class. A simple function that does one thing is often better than a class:

```
def calculate_area(radius):  
    return 3.14 * radius ** 2
```

There is no need for a class here - the function does one thing and contains no data.

UML notation

As we now begin to build classes, it can be useful to have a visual tool for communicating to others how we think of a class, including its methods and attributes. UML - Unified Modeling Language - is a useful tool for this. We will return to it in the chapter on advanced classes, Chapter 10.

8.7 A more advanced class: Account

We will now build an `Account` class representing a bank account, and extend it step by step. Each step introduces new concepts:

- Basic class with methods `deposit()`, `withdraw()` etc
- A separate `Transaction` class to store transactions

- Properties for access control
- Class variables and class methods

8.7.1 Basic Account class

```

1: # file: sc_08_02_Account1.py
2: class Account:
3:     def __init__(self, cust_id, account_no, start_balance,
interest):
4:         self._cust_id    = cust_id
5:         self._account_no = account_no
6:         self._balance    = start_balance
7:         self._interest   = interest
8:
9:     def get_balance(self):
10:        return self._balance
11:
12:     def set_balance(self, new_balance):
13:        self._balance = new_balance
14:
15:     def deposit(self, amount):
16:        if amount > 0:
17:            self._balance += amount
18:        return self._balance
19:
20:     def withdraw(self, amount):
21:        if amount <= self._balance:
22:            self._balance -= amount
23:        return self._balance
24:
25:     def add_monthly_interest(self):
26:        self._balance += self.calculate_monthly_interest()
27:
28:     def calculate_monthly_interest(self):
29:        return self._balance * self._interest / 100 / 12
30:
31:     def __str__(self):
32:        return (f"Customer id    = {self._cust_id}\n"
33:               f"Account no     = {self._account_no}\n"
34:               f"Balance        = {self._balance:.2f}\n"
35:               f"Interest       = {self._interest}%")

```

Code comments:

Line 3-7

`__init__()` receives customer ID, account number, starting balance and rate, and stores them as attributes. Note that interest is given as an integer -

a 5% rate is stored as 5. All attributes have a leading underscore, signalling that client code should not change them directly.

Line 9-13

`get_balance()` and `set_balance()` are traditional getter/setter methods. We will replace them with properties in the next version.

Line 15-23

`deposit()` and `withdraw()` perform simple validations: deposits must be positive, withdrawals cannot exceed the balance. Both return the new balance.

Line 28-29

We divide by 100 to convert from a percentage to a decimal, and by 12 to get the monthly interest rate.

Line 31-35

`__str__()` is called implicitly when we pass the object to `print()`. If the method is missing, Python uses `__str__()` from the ultimate base class, which just gives a memory address. Try commenting out the `__str__()` method here, and see what happens.

Client code:

```
1: # file: sc_08_03_use_Account1_.py
2: from sc_08_02_Account1 import Account
3:
4: account1 = Account(1, 1000, 50000, 7)
5: account2 = Account(2, 1001, 10000, 5)
6: print(account1)
7: print(account2)
8: Account.deposit(account1, 500)
9: print(f'New balance after deposit:{account1.get_balance()}')
10: account1.withdraw(1000)
11: print(f'New balance after withdr.:{account1.get_balance()}')
12: account1.add_monthly_interest()
13: print(f'New balance after monthly interest:
{account1._balance}')
```

Line 6-7

`print()` finds `__str__()` in `Account` and calls it implicitly.

Line 13

We access `_balance` directly -, even if we have marked the attribute as private - Python does not stop us. The underscore is just a convention saying “stay away (use `get_balance()` instead)”.

Output from the client code:

```
Customer id = 1
Account no = 1000
Balance = 50000
Interest = 7

Customer id = 2
Account no = 1001
Balance = 10000
Interest = 5

New balance after deposit: 50500
New balance after withdr.: 49500
New balance after monthly interest: 49788.75
```

8.8 Python has no private data members

Programmers from C++ and Java are used to attributes being declared private so that client code cannot access them directly. Python *does not* have this mechanism - all attributes and methods are accessible from outside.

The convention in Python is that a single leading underscore signals "internal use - keep away". It is an agreement between developers, not a technical barrier. Python is built on trust between developers rather than enforced rules.

Discussion / dilemma: when should we use a leading underscore?

As mentioned - in C++ and Java there are clear rules: `private` means inaccessible from outside, `public` means open. Python does not have this distinction, so the convention is a little more nuanced:

Use a leading underscore (`_name`) when the attribute is an implementation detail - something client code should not deal with directly. This applies especially when you plan to add validation via properties, or when the value should not be freely changed from outside.

Omit the underscore when the attribute is meant to be directly accessible and you do not need to control access. This is perfectly Pythonic for simple data classes where openness is a deliberate design decision.

In practice, many Python developers choose to use underscore consistently on all instance attributes - it is a safe default that makes it easy to add properties later without changing the client code.

Under the hood: name mangling with double underscore

You can start an attribute name with a double underscore, e.g. `self.__balance`. Python then renames it internally to `_Account__balance` - this is called name mangling. The purpose is to avoid name conflicts in inheritance, not to create truly private attributes. The attribute is still accessible if you know the internal name. *Name mangling is not recommended as a replacement for the private mechanism from C++/Java.*

8.9 Account with transaction log

We introduce a Transaction class to log what happens on the account. This is a good example of classes collaborating - Account creates Transaction objects and stores them in a list.

```
# file: sc_08_04_Account2.py
from datetime import datetime

class Transaction:
    def __init__(self, amount, trans_type):
        self._amount = amount
        self._trans_type = trans_type # "deposit", "withdraw",
        "interest"
        self._timestamp = datetime.now()

    def __str__(self):
        return (f"{self._timestamp:%Y-%m-%d %H:%M:%S} | "
                f"{self._trans_type.capitalize():8} | "
                f"{self._amount:8.2f}")
```

Transaction is deliberately simple - it stores data and knows how to print itself. It is Account's responsibility to create and store transactions, that is, objects of this class (Transactions).

In Account we add a list for transactions and update the methods that change the balance. Here we show only the parts that have changed:

```
class Account:
    def __init__(self, cust_id, account_no, start_balance,
        interest):
        self._cust_id = cust_id
        self._account_no = account_no
        self._balance = start_balance
```

```

        self._interest      = interest
        self._transactions = []      # NEW

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            self._transactions.append(Transaction(amount,
"deposit")) # NEW
        return self._balance

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
            self._transactions.append(Transaction(amount,
"withdraw")) # NEW
        return self._balance

    def add_monthly_interest(self):
        monthly_interest = self.calculate_monthly_interest()
        self._balance     += monthly_interest
        self._transactions.append(Transaction(monthly_interest,
"interest")) # NEW

    def print_transactions(self):
        print("Date and time          | Type          | Amount")
        print("-" * 42)
        for trans in self._transactions:
            print(trans)

```

Note that `print(trans)` calls `Transaction.__str__()` implicitly - exactly the same as happens when we print an `Account` object.

Client code

```

# file: sc_08_05_use_Account2.py
from sc_08_04_Account2 import Account

account = Account(1, 1000, 50000, 7.0)

account.deposit(2000)
account.withdraw(500)
account.add_monthly_interest()

account.print_transactions()
print(account)

```

Output

Date and time	Type	Amount
2026-05-07 21:38:42	Deposit	2000.00
2026-05-07 21:38:42	Withdraw	500.00
2026-05-07 21:38:42	Interest	300.42

Customer id = 1
 Account no = 1000
 Balance = 51800.42
 Interest = 7.0%

8.10 Properties for access control

Properties are the most Pythonic way to protect attributes. They allow us to control reading and writing, add validation, and make attributes read-only - without the client code needing to change.

With properties, client code accesses the attribute by its "clean" name - `account1.balance` - without dealing with the internal underscore representation. Inside the class we still use `self._balance`. Properties thus hide the implementation detail from the client code, as it should be.

The recipe for a property with validation:

1. The attribute follows the leading underscore convention:
`self._balance`
2. A getter method decorated with `@property` - called implicitly when the attribute name appears on the right side of "=" in client code
3. A setter method decorated with `@attribute_name.setter` - called implicitly when the attribute name appears on the left side of "=" in client code

Fact: what is a decorator?

`@property` and `@balance.setter` are examples of decorators - they start with `@` and are placed directly above a function or method definition. A decorator is a way to "wrap" or extend a function with extra behaviour, without changing the function itself.

In this case `@property` makes Python treat the method as an attribute rather than a regular method call - that is why we can write `account1.balance` instead of `account1.balance()`.

We will look more closely at how decorators work in a later chapter. For now it is enough to know that `@property` and `@name.setter` are the recipe for creating properties in Python.

```
# file: sc_08_06_Account3.py (extract, selected parts)

class Account:
    # ... __init__ & other methods unchanged

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, new_balance):
        if new_balance < 0:
            print("Balance cannot be negative!")
        else:
            self._balance = new_balance

    @property
    def interest(self):
        return self._interest

    @interest.setter
    def interest(self, new_interest):
        if new_interest < 0:
            print("Interest rate cannot be negative!")
        else:
            self._interest = new_interest
```

The client code can now look like this:

```
# file: sc_08_07_use_Account3.py
from sc_08_06_Account3 import Account

account1 = Account(123, 456, 1000, 2.5)
account1.deposit(500)
account1.deposit(1500)
account1.withdraw(200)
account1.withdraw(2000)account1.add_monthly_interest()
account1.print_transactions()

# Using properties
print(f"\nBalance after: {account1.balance:.2f}")
print(f"Interest rate: {account1.interest}%")

# Change balance and interest via properties
account1.balance = 2000
account1.interest = 3.0
print(f"\nNew balance: {account1.balance:.2f}")
```

The big gain is that the setter method can contain logic and validation. The client code does not see this - it just writes `account1.balance = value` regardless.

Output

Date and time	Type	Amount
2026-05-07 21:51:29	Deposit	500.00
2026-05-07 21:51:29	Deposit	1500.00
2026-05-07 21:51:29	Withdraw	200.00
2026-05-07 21:51:29	Withdraw	2000.00
2026-05-07 21:51:29	Interest	1.67

Balance after: 801.67
Interest rate: 2.5%

New balance: 2000.00

get_balance() or @property?

We replaced `get_balance()` with a property. When should you choose which?

Use a property when access should look like ordinary attribute reading - it gives more readable and Pythonic code. Use an explicit get method

when the operation is expensive (e.g. a database lookup) and you want the call to look like a function call, not a variable read.

Tip: when should you use properties?

- Use a property when you want to validate or restrict what can be set.
- Use a property when you want to hide internal implementation.
- Use a property for read-only attributes (getter only, no setter).
- Use a property for computed values derived from other attributes.
- Do not use a property for simple attributes without extra logic - plain attributes are simplest.

8.10.1 Another property example: Temperature

Let us look at another example where properties give a clear benefit. The class `Temperature` represents a temperature. We want the user to be able to read and set the temperature in both Celsius and Fahrenheit - and we want to guard against physically impossible values. Internally the class stores only one value: `_celsius`. Everything else is properties that compute from this.

Three properties with different roles:

- `celsius` - read and set the temperature in Celsius, with validation against absolute zero.
- `fahrenheit` - read and set the temperature in Fahrenheit. Internally the value is always stored as Celsius, so the conversion happens automatically.
- `kelvin` - read-only. Kelvin is uniquely determined by the Celsius value and can never be negative. A setter would be meaningless.

```
# file: sc_08_17_temperature.py
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius    # uses the setter,
                                # validation happens here

    @property
    def celsius(self):
        return self._celsius
```

```

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError(f"Temperature {value} is below
absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self.celsius = (value - 32) * 5 / 9 # uses the celsius
                                           # setter, validation
                                           # for free

    @property
    def kelvin(self):
        # no setter - read-only
        return self._celsius + 273.15

    def __str__(self):
        return f"{self.celsius} C | {self.fahrenheit:.1f} F |
{self.kelvin:.2f} K"

if __name__ == "__main__":
    t = Temperature(25)
    print(t)           # 25 C | 77.0 F | 298.15 K
    t.fahrenheit = 32
    print(t.celsius)   # 0.0
    # t.kelvin = 300    # AttributeError: read-only
    # t.celsius = -300  # ValueError: below absolute zero

```

Notice a couple of elegant details. In the constructor we write `self.celsius = celsius`, not `self._celsius = celsius`. This means the setter runs already when the object is created - validation is guaranteed from the very first moment, not only when someone explicitly sets a new value.

The Fahrenheit setter calls `self.celsius = ...` instead of `self._celsius = ...`. This means the Celsius setter's validation runs automatically even when someone sets the temperature via Fahrenheit. We avoid duplicating the validation logic.

The class has one internal value (`_celsius`), but exposes three interfaces externally. The client code chooses whichever scale is most natural - the class handles the rest.

Output

```
25 C | 77.0 F | 298.15 K
0.0
```

8.11 dataclass - when the class is just data

Some classes are simple data containers - they store some values together and may need comparison and readable output, but no significant logic. Let us look at a `Point` class representing a point in a coordinate system:

```
# file: sc_08_15_point_manual.py
class Point:
    def __init__(self, x, y):
        self._x = x
        self._y = y

    def __str__(self):
        return f"Point(x={self._x}, y={self._y})"

    def __eq__(self, other):
        return self._x == other._x and self._y == other._y

p1 = Point(3, 4)
p2 = Point(3, 4)
p3 = Point(1, 2)
print(p1)           # Point(x=3, y=4)
print(p1 == p2)     # True
print(p1 == p3)     # False
```

This is straightforward code, but we write a lot to achieve little. Python has a built-in mechanism for exactly this situation - `@dataclass`:

```
# file: sc_08_15_point_dataclass.py
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p1 = Point(3, 4)
p2 = Point(3, 4)
p3 = Point(1, 2)
```



```
print(p1)           # Point(x=3, y=4)
print(p1 == p2)     # True
print(p1 == p3)     # False
```

The two versions are functionally equivalent. `@dataclass` is a decorator (we look at decorators more closely in a later chapter) that reads the field declarations and *generates* `__init__()`, `__repr__()` and `__eq__()` *automatically*. We do not have to write them by hand.

Fields are declared with name and type directly in the class block: `x: int` means the class has a field `x` of type `int`. Note that these are plain attributes without a leading underscore - `dataclass` exposes them directly, without properties.

We can also give fields default values:

```
@dataclass
class Point:
    x: int = 0
    y: int = 0

origo = Point()
print(origo)           # Point(x=0, y=0)
```

8.11.1 Default values for mutable fields

The default value syntax we just saw - `x: int = 0` - works for *immutable* values like numbers and strings. But consider a `Bag` class that should hold a list of items, where the default is an empty list:

```
@dataclass
class Bag:
    items: list = [] # ValueError - do not do this
```

Python raises a `ValueError` immediately when the class is defined. The reason is that `[]` is evaluated once, when the class is created - not once per instance. All `Bag` objects would therefore share the exact same list object. Modifying one instance would silently affect all others:

```
# If Python allowed this - which it does not - the result would
# be:
b1 = Bag()
b2 = Bag()
b1.items.append("apple")
print(b2.items) # ['apple'] - b2 is affected too, which is
# wrong
```

This problem is not unique to `@dataclass` - it is a general Python pitfall with mutable default arguments, which we have seen in functions as well. `@dataclass` simply catches it early and refuses, rather than letting the bug appear silently at runtime.

The solution is `field()` with `default_factory`:

```
from dataclasses import dataclass, field

@dataclass
class Bag:
    items: list = field(default_factory=list)

b1 = Bag()
b2 = Bag()
b1.items.append("apple")
print(b1.items)    # ['apple']
print(b2.items)    # [] - separate list, not shared
```

`field()` is a helper function that gives more control over individual fields. It has several parameters - the most important one here is `default_factory`.

The name describes what it does: it is a factory for default values. Instead of giving a ready-made value as the default, we give a function that creates a new value each time. `@dataclass` calls this function automatically when a new instance is created:

```
field(default_factory=list)
# roughly equivalent to:
# for each new object → items = list() → []
```

We pass in `list` not `list()`. We are giving the function itself, not the result of calling it. `@dataclass` calls it for us, once per new instance, so that each object gets its own fresh list.

`default_factory` can take any function with no arguments - typically `list`, `dict` or `set` for empty collections, but also custom functions if more complex default values are needed.

Fact: `field()` and mutable defaults

`default_factory` solves the mutable default problem, but `field()` can do more.

Some useful parameters:

`default_factory=list` - call `list()` for each new instance instead of sharing one object

`repr=False` - exclude the field from `__repr__()` output, useful for large or sensitive values

`compare=False` - exclude the field from `__eq__()` comparisons

`init=False` - do not include the field as a constructor parameter; the field gets its value from default or `default_factory` only

A field marked `init=False` with a `default_factory` is useful for computed or cached state that should not be passed in by the caller.

What dataclass gives us - and what it does not

`@dataclass` automatically generates `__init__()`, `__repr__()` and `__eq__()`. With additional arguments we can also get `__lt__()`, `__gt__()` and similar (`@dataclass(order=True)`), and frozen (immutable) objects (`@dataclass(frozen=True)`).

What `@dataclass` does not give us is validation, properties and custom logic. If we want to prevent `x` from being set to a negative number, or compute the distance to the origin as a property, we have to add it ourselves - and then we are essentially back to a regular class. Account from this chapter would not have been simpler with `@dataclass` - the logic requires full control over the constructor and property setters.

Nice to know: type hints in dataclass

`@dataclass` requires type-annotated fields - `x: int` is not optional. This makes `@dataclass` a good argument for using type hints consistently: we get functionality for free as a reward.